

Documentation complète — HealthAI Coach (Flutter)

Projet : MSPR TP502

Application : healthai_coach_mobile

Framework : Flutter 3.x / Dart SDK ≥ 3.0

Version : 1.0.0+1

Date : Avril 2026

Table des matières

Partie I — Documentation technique

1. [Architecture générale](#)
2. [Stack technique](#)
3. [Structure des fichiers](#)
4. [Couche Core](#)
5. [Feature : Authentification](#)
6. [Feature : Nutrition](#)
7. [Feature : Menu IA \(Spoonacular\)](#)
8. [Feature : Coach](#)
9. [Navigation](#)
10. [Thème et design system](#)
11. [Communication avec le backend](#)
12. [État des connexions backend](#)
13. [Ce qui reste à faire](#)

Partie II — Accessibilité et adoption utilisateur

1. [Positionnement et public cible](#)
 2. [Choix structurels pour l'accessibilité](#)
 3. [Accessibilité visuelle](#)
 4. [Accessibilité motrice](#)
 5. [Accessibilité cognitive](#)
 6. [Accessibilité des features spécifiques](#)
 7. [Accompagnement à l'adoption](#)
 8. [Limites identifiées et axes d'amélioration](#)
-

Partie I — Documentation technique

1. Architecture générale

L'application suit une **Clean Architecture** organisée par feature. Chaque feature est découpée en trois couches indépendantes :

```
feature/  
├─ domain/           # Entités métier + interfaces de repository (pur Dart, 0 dépendance)  
├─ data/             # Implémentations concrètes (modèles JSON, appels API)  
└─ presentation/     # UI, providers Riverpod, widgets
```

Pourquoi ce choix ?

Cette séparation garantit que la logique métier ne dépend jamais du framework UI ni du transport réseau. Si l'on change Dio pour un autre client HTTP, ou Riverpod pour un autre state manager, seule la couche `data` ou `presentation` est impactée. C'est aussi ce qui permet de brancher facilement un mock ou une vraie implémentation sans toucher à l'UI.

2. Stack technique

Domaine	Librairie	Version	Justification
State management	<code>flutter_riverpod</code>	<code>^2.5.0</code>	<code>AsyncNotifier</code> couvre nativement loading/error/data sans boilerplate
Navigation	<code>go_router</code>	<code>^14.0.0</code>	Navigation déclarative, deep links, type-safe routes
HTTP client	<code>dio</code>	<code>^5.4.0</code>	Intercepteurs pour auth token, gestion d'erreurs centralisée
Auth locale	<code>shared_preferences</code>	<code>^2.3.0</code>	Persistance simple du JWT et userId entre sessions
Caméra	<code>camera</code> + <code>image_picker</code>	<code>^0.11 / ^1.1</code>	<code>camera</code> pour le viewfinder live, <code>image_picker</code> pour la galerie
Compression image	<code>flutter_image_compress</code>	<code>^2.3.0</code>	Réduit le poids des photos avant upload (réseau + performance)

Domaine	Librairie	Version	Justification
Images réseau	cached_network_image	^3.3.0	Cache automatique des photos de recettes Spoonacular
Stockage offline	isar	^3.1.0	Base de données locale NoSQL rapide pour le plan d'entraînement hors ligne
Vidéo	video_player	^2.8.0	Démos vidéo des exercices (feature Coach)

3. Structure des fichiers

```
lib/
├─ app/
│   ├── app.dart          # Point d'entrée app, router GoRouter
│   └─ theme.dart         # Design system (couleurs, thème Material3)
│
├─ core/
│   ├── auth/
│   │   └─ token_storage.dart    # Persistance JWT + userId (SharedPreferences)
│   ├── constants/
│   │   └─ api_constants.dart    # Toutes les URLs d'endpoints
│   ├── network/
│   │   └─ dio_client.dart       # Instance Dio singleton + intercepteur auth
│   ├── errors/
│   │   └─ failures.dart        # Classes d'erreurs métier
│   └─ utils/
│       └─ image_utils.dart      # Compression et nettoyage des images temp
│
├─ features/
│   ├── auth/
│   │   └─ presentation/screens/login_screen.dart
│   ├── nutrition/
│   │   ├── domain/
│   │   │   ├── entities/meal_analysis.dart
│   │   │   └─ repositories/nutrition_repository.dart
│   │   └─ data/
│   │       └─ models/meal_analysis_model.dart
```

```

| | | └─ repositories/nutrition_repository_impl.dart
| | └─ presentation/
| |   │ └─ providers/nutrition_provider.dart
| |   │ └─ screens/camera_screen.dart
| |   │ └─ screens/meal_result_screen.dart
| |   └─ widgets/macro_table_widget.dart
| └─ meal_plan/
|   │ └─ domain/
|   │   │ └─ entities/meal_plan.dart
|   │   │ └─ repositories/meal_plan_repository.dart
|   │   └─ data/
|   │     │ └─ models/meal_plan_model.dart
|   │     │ └─ repositories/meal_plan_repository_impl.dart
|   │     └─ presentation/
|   │       │ └─ providers/meal_plan_provider.dart
|   │       └─ screens/meal_plan_screen.dart
|   └─ coach/
|     │ └─ domain/entities/      # SessionState, SessionFeedback
|     │ └─ data/repositories/    # CoachRepositoryMock (temporaire)
|     │ └─ presentation/
|     │   │ └─ providers/coach_provider.dart
|     │   └─ screens/           # CoachHubScreen, SessionScreen, RpeScreen
|   └─ offline/
|     │ └─ domain/entities/      # Exercise, DayPlan, WeekPlan
|     │ └─ data/                 # Modèles Isar + OfflineRepositoryMock
|     └─ presentation/          # WeekPlanScreen, providers
|
└─ shared/
  └─ screens/
    │ └─ home_screen.dart        # Shell avec BottomNavigationBar (4 tabs)
    └─ dev_home_screen.dart     # Écran de debug (non exposé en prod)

```

4. Couche Core

4.1 ApiConstants

Fichier : `lib/core/constants/api_constants.dart`

Centralise toutes les URLs. La base URL est injectable via `--dart-define` pour distinguer les environnements :

```
static const String baseUrl = String.fromEnvironment(
  'BFF_BASE_URL',
  defaultValue: 'http://localhost:3002',
);
```

Choix : utiliser `String.fromEnvironment` plutôt qu'un fichier `.env` tiers permet de compiler la valeur dans le binaire Flutter sans dépendance supplémentaire.

Constante	Endpoint	Usage
<code>signIn</code>	<code>POST /api/auth/sign-in/email</code>	Connexion Better Auth
<code>signUp</code>	<code>POST /api/auth/sign-up/email</code>	Inscription
<code>uploadMeal</code>	<code>POST /api/nutrition/upload</code>	Upload photo repas
<code>analyzeMeal</code>	<code>POST /api/nutrition/analyze</code>	Analyse IA macros (gRPC engine-go)
<code>generateMenu</code>	<code>POST /api/generate-menu</code>	Menu Spoonacular
<code>weeklyPlan</code>	<code>GET /api/coach/plan</code>	Plan coach (non implémenté BFF)
<code>userProfile</code>	<code>GET /api/user/profile</code>	Profil utilisateur

4.2 DioClient

Fichier : `lib/core/network/dio_client.dart`

Instance Dio singleton avec : - `baseUrl` depuis `ApiConstants` - `connectTimeout` : 10 secondes - `receiveTimeout` : 30 secondes - `LogInterceptor` : log toutes les requêtes/réponses en debug - `_AuthInterceptor` : injecte le Bearer token sur chaque requête

Fonctionnement de `_AuthInterceptor` : 1. Avant chaque requête → lit le token depuis `TokenStorage` → injecte `Authorization: Bearer <token>` 2. Sur erreur 401 → efface le token automatiquement (déconnexion silencieuse)

Pourquoi un singleton ?

Dio maintient un pool de connexions. Instancier Dio à chaque appel recréerait ce pool inutilement. Le singleton garantit une seule instance partagée par toutes les features.

4.3 TokenStorage

Fichier : `lib/core/auth/token_storage.dart`

Persistence du JWT et du `userId` Better Auth via `SharedPreferences` .

Clés stockées :

- `auth_token` → JWT Bearer token
- `auth_user_id` → ID utilisateur (UUID Better Auth)

Pourquoi stocker les deux ?

Le `userId` est nécessaire lors de l'upload nutrition (`POST /api/nutrition/upload`) pour associer le repas à l'utilisateur en base MongoDB. Sans lui, on retombait sur un `'user_placeholder'` hardcodé.

5. Feature : Authentification

Flux de connexion

```
LoginScreen
  → saisie email + password
  → POST /api/auth/sign-in/email (Better Auth / backend-hono)
  → réponse : { token: "...", user: { id: "..." } }
  → TokenStorage.save(token, userId: userId)
  → context.go('/home')
```

Gestion d'erreurs

Les erreurs `DioException` sont catchées et affichées inline dans l'écran (pas de `SnackBar`) via `_errorMessage` .
Le message provient du champ `message` ou `error` de la réponse JSON du BFF.

Limites actuelles

- L'inscription (`S'inscrire`) et la récupération de mot de passe sont des boutons présents dans l'UI mais sans navigation ni appel API branché.
- L'authentification sociale (Google, Apple) est affichée mais non implémentée.
- Pas de bouton de déconnexion dans l'UI — la déconnexion se produit automatiquement si le serveur retourne 401.

6. Feature : Nutrition

6.1 Flux complet

```

CameraScreen
├─ Prise de photo (CameraController)
├─ Sélection galerie (ImagePicker)
└─
    ↓
    ImageUtils.compress()           ← compression JPEG avant upload
    ↓
    POST /api/nutrition/upload      ← multipart/form-data { file, userId }
    → BFF transmet à python.medev-tech.fr/predict/upload
    → réponse : { macros: { calories, protein, carbs, fat }, detectedFood }
    ↓
    MealAnalysisModel.fromUploadJson()
    ↓
    POST /api/nutrition/analyze     ← { ingredients, macros } – NON BLOQUANT
    → réponse : { is_safe, warnings, advice }
    ↓
    model.withAiAnalysis(...)
    ↓
    MealResultScreen               ← affiche MacroTableWidget + _AiAnalysisPanel
  
```

Choix clé — appel IA non bloquant : L'endpoint `/api/nutrition/analyze` passe par gRPC → engine-go qui peut être instable. Si cet appel échoue, les macros brutes de l'upload sont quand même affichées. L'erreur est loggée (`debugPrint`) mais ne fait pas crasher le flux utilisateur.

6.2 Entités domaine

FoodItem : représente un aliment individuel avec ses macros (calories, protéines, glucides, lipides).

MealAnalysis : agrège une liste de `FoodItem`, les totaux calculés, des suggestions textuelles, et les champs IA optionnels (`isSafe`, `warnings`, `advice`).

6.3 MacroTableWidget

Widget central de `MealResultScreen`. Il propose deux modes :

- **Lecture** : `DataTable` avec les macros par aliment + ligne Total

- **Édition (Corriger)** : chaque cellule devient un `TextFormField` permettant de corriger les valeurs si l'IA s'est trompée. La validation recalcule les totaux et met à jour le provider.

Ce mode édition répond à un besoin concret : si la reconnaissance visuelle retourne des valeurs approximatives, l'utilisateur peut les corriger manuellement sans quitter l'application.

6.4 `_AiAnalysisPanel`

Panel conditionnel affiché uniquement si `analysis.isSafe != null` (c'est-à-dire si l'appel à `/api/nutrition/analyze` a réussi). Affiche : - Badge vert `Repas sûr` ou rouge `Attention requise` - Liste des avertissements (ex: "Repas calorique > 800 kcal") - Conseil personnalisé

6.5 Service Python de reconnaissance visuelle

Le service `python.medev-tech.fr` est opérationnel. Il expose `POST /predict/upload` et retourne la classe de l'aliment avec un score de confiance et une estimation calorique.

Exemple de réponse sur une photo de donuts :

```
{
  "top_prediction": { "class_name": "donuts", "score": 0.9837 },
  "calories": {
    "top1": {
      "kcal_per_100g": 452.0,
      "portion_g": 70.0,
      "estimated_kcal": 316.4
    }
  }
}
```

Intégration Flutter requise (une ligne, dans `meal_analysis_model.dart`) :

```
name: (json['detectedFood'] as String?) ?? 'Repas analysé',
```

Le reste du flux est déjà câblé.

7. Feature : Menu IA (Spoonacular)

7.1 Objectif

Permettre à l'utilisateur de générer un menu personnalisé (Déjeuner + Dîner) en fonction de ses préférences caloriques, son objectif et ses allergies. C'est la feature IA **entièrement fonctionnelle et en production** de l'application.

7.2 Flux

```
MealPlanScreen
  → slider calorique (1200-3500 kcal, pas de 100)
  → chips objectif : weight_loss / muscle_gain / maintenance
  → chips allergies : gluten, dairy, nuts, eggs, soy, fish
    ↓
  Bouton "Générer mon menu"
    ↓
  MealPlanNotifier.generate(calories, allergies, goal)
    ↓
  POST /api/generate-menu
  body : { user: { dailyCaloriesTarget, allergies, goal } }
    ↓
  réponse : { lunch: { ...recipe }, dinner: { ...recipe }, totalCalories }
    ↓
  _MealPlanResult → 2x _RecipeCard
```

7.3 Entités domaine

MealPlanRecipe : recette complète avec id, name, ingredients, calories, protein, carbs, fat, sugar, sodium, pricePerServing, preparationTime, imageUrl, diets, dishTypes, score.

GeneratedMealPlan : conteneur de `lunch` + `dinner` + `totalCalories`.

7.4 Affichage

Chaque `_RecipeCard` affiche : - Photo de la recette via `CachedNetworkImage` (avec placeholder et fallback icône) - Badge de label (DÉJEUNER / DÎNER) en overlay sur la photo - Badge score étoile (si score > 0) - Chips macros : kcal, protéines, glucides (couleurs codées) - Temps de préparation + prix par personne - Tags régimes alimentaires (vegan, gluten-free, etc.) - Ingrédients (8 premiers + compteur du reste)

7.5 MealPlanNotifier

`AsyncNotifier<GeneratedMealPlan?>` — état initial `null`. Lors de `generate()`, l'état passe à `AsyncLoading` puis `AsyncData` ou `AsyncError`. Le bouton de génération est désactivé pendant le chargement.

8. Feature : Coach

8.1 État actuel

La feature Coach est **partiellement implémentée**. L'UI est complète et fonctionnelle avec des données mockées. La connexion au backend est bloquée pour des raisons indépendantes du Flutter (voir section 12).

8.2 Ce qui fonctionne

- `CoachHubScreen` : affiche la séance du jour, les stats de la semaine, le planning 7 jours
- `SessionScreen` : chronomètre de séance, navigation entre exercices, phase repos
- `RpeScreen` : saisie du ressenti d'effort post-séance (RPE 1–10)
- `SessionNotifier` : gestion complète du timer, pause, passage à l'exercice suivant

8.3 Données actuellement mockées

Le planning hebdomadaire est défini dans `coach_hub_screen.dart` via `_mockWeek` (constante Dart, 3 jours). `CoachRepositoryMock` simule un délai réseau de 800ms pour `sendFeedback`.

8.4 Architecture de connexion prévue

```
CoachRepository (interface)
└─ sendFeedback(SessionFeedback) → bool

À compléter :
└─ getWeekPlan(String userId) → WeekPlan ← méthode manquante
```

9. Navigation

Fichier : `lib/app/app.dart`

Routeur GoRouter avec 8 routes déclaratives :

Route	Widget	Paramètre
<code>/login</code>	<code>LoginScreen</code>	—
<code>/home</code>	<code>HomeScreen</code>	—
<code>/camera</code>	<code>CameraScreen</code>	—

Route	Widget	Paramètre
/meal-result	MealResultScreen	extra: String (imagePath)
/week-plan	WeekPlanScreen	—
/session	SessionScreen	extra: DayPlan
/rpe	RpeScreen	—
/meal-plan	MealPlanScreen	—

HomeScreen est le shell principal avec `IndexedStack` (3 écrans persistants) et `BottomNavigationBar` à 4 items :

Index tap	Action
0	Affiche <code>NutritionHubScreen</code>
1	<code>context.push('/camera')</code> (bouton central +)
2	Affiche <code>CoachHubScreen</code>
3	Affiche <code>MealPlanScreen</code>

Pourquoi `IndexedStack` ?

Contrairement à `PageView`, `IndexedStack` garde les widgets en mémoire entre les onglets. L'état des providers Riverpod n'est pas réinitialisé quand l'utilisateur change d'onglet.

Pourquoi `GoRouter` ?

Permet de passer des objets complexes entre routes (`extra: DayPlan`, `extra: imagePath`) sans les sérialiser dans l'URL.

10. Thème et design system

Fichier : `lib/app/theme.dart`

Design épuré inspiré d'applications wellness (fond beige chaud, vert primaire, accents ambre).

Palette `AppColors`

Token	Hex	Usage
primary	#1D9E75	Actions principales, icônes actives
primaryDark	#0F6E56	Hover, focus
primaryLight	#E1F5EE	Backgrounds de chips, badges
accent	#FAC775	Indicateurs, streak
background	#F1EFE8	Fond général (beige chaud)
surface	#FFFFFF	Cards, bottom nav
surfaceAlt	#E8E7E0	Champs de saisie, placeholders
textPrimary	#444441	Texte principal
textSecondary	#888780	Labels, sous-titres
textTertiary	#B4B2A9	Placeholders, métadonnées
border	#D3D1C7	Bordures de cards

Material 3

Le thème utilise `useMaterial3: true` . Les composants sont tous redéfinis globalement (`ElevatedButton` , `OutlinedButton` , `InputDecoration` , `BottomNavigationBar` , `Chip`) pour éviter toute surcharge locale dans les widgets.

11. Communication avec le backend

Architecture globale



- └ Spoonacular API → Génération menu IA
- └ gRPC → engine-go → Analyse nutritionnelle, workout
- └ HTTP → python.medev-tech.fr → Reconnaissance visuelle aliments (opérationnel)

Authentification

Better Auth retourne { token, user: { id } } à la connexion. Le token JWT est persisté en local et injecté automatiquement sur toutes les requêtes par _AuthInterceptor . Sur 401, le token est effacé.

Format des requêtes

- Upload nutrition : multipart/form-data avec file (image compressée) et userId
- Toutes les autres : application/json
- Header commun : Authorization: Bearer <token>

12. État des connexions backend

Feature Flutter	Endpoint BFF	Statut	Détail
Connexion	POST /api/auth/sign-in/email	✓ Fonctionnel	Better Auth opérationnel
Upload photo nutrition	POST /api/nutrition/upload	✓ Fonctionnel	Macros retournées par le BFF
Reconnaissance visuelle	python.medev-tech.fr/predict/upload	✓ Opérationnel	Modèle déployé, testé (98.4% sur donuts)
Analyse IA nutrition	POST /api/nutrition/analyze	⚠ Partiel	gRPC engine-go instable — non bloquant
Génération menu	POST /api/generate-menu	✓ Fonctionnel	Spoonacular, données réelles
Plan coach	GET /api/coach/plan	✗ Non connecté	Endpoint inexistant dans BFF
Profil utilisateur	GET /api/user/profile	✗ Non connecté	Endpoint disponible, non branché Flutter

13. Ce qui reste à faire

Priorité haute

Intégration reconnaissance visuelle (nutrition)

Le service Python est opérationnel. Une seule ligne Flutter à modifier (`meal_analysis_model.dart`) pour afficher le nom de l'aliment détecté une fois que le BFF transmet `detectedFood` :

```
name: (json['detectedFood'] as String?) ?? 'Repas analysé',
```

Connexion feature Coach

Nécessite côté backend : implémenter `GET /api/coach/plan` + seeder la table exercices.

Côté Flutter : ajouter `getWeekPlan()` à `CoachRepository`, créer `CoachRepositoryImpl`, remplacer `_mockWeek`.

Priorité moyenne

- **Écran profil** : brancher `GET /api/user/profile` via un provider dédié
- **Déconnexion manuelle** : ajouter un bouton logout appelant `TokenStorage.clear()`
- **Inscription** : créer `SignUpScreen` appelant `POST /api/auth/sign-up/email`

Priorité basse

- **Historique nutrition** : brancher `GET /api/nutrition/history` dans `NutritionHubScreen`
- **Plan semaine offline** : connecter `OfflineRepositoryImpl` (Isar déjà configuré)
- **Feedback séance** : remplacer `'test-user-001'` par `await TokenStorage.getUserId()`

Partie II — Accessibilité et adoption utilisateur

14. Positionnement et public cible

HealthAI Coach s'adresse à un public large : personnes souhaitant suivre leur alimentation, leurs séances sportives et leur santé en général. Ce public inclut des utilisateurs sans expertise numérique particulière, des personnes en situation de handicap visuel partiel, et des utilisateurs en mobilité (lors d'une séance de sport, les mains chargées, etc.).

L'accessibilité n'est pas une option dans ce contexte : une application de santé doit être utilisable par le plus grand nombre, notamment parce que les personnes les plus susceptibles d'en avoir besoin (personnes âgées, personnes

en réhabilitation, personnes avec restrictions alimentaires médicales) sont souvent celles qui ont des contraintes d'accessibilité.

15. Choix structurels pour l'accessibilité

15.1 Material Design 3 comme socle

Choix : l'application utilise `useMaterial3: true` dans son thème Flutter.

Pourquoi : Material 3 intègre nativement les recommandations WCAG. En adoptant M3, l'application bénéficie automatiquement de : - Rôles sémantiques sur tous les composants natifs (`ElevatedButton` , `Slider` , `Chip` , `DataTable`) - Gestion correcte du focus clavier - Tailles de cibles tactiles respectant les recommandations (minimum 48×48 dp) - Support natif des lecteurs d'écran (TalkBack Android, VoiceOver iOS)

Les composants Material sont reconnus par les technologies d'assistance : un `ElevatedButton` est annoncé comme "bouton" par TalkBack sans configuration supplémentaire.

15.2 Thème centralisé — aucune couleur hardcodée dans les widgets

Choix : toutes les couleurs passent par `AppColors` . Aucun widget n'utilise une couleur RGB directe pour les éléments fonctionnels.

Pourquoi : en centralisant les couleurs dans un design token, l'ajout d'un thème sombre ou d'un mode haute-contraste ne nécessite que la modification de `AppColors` , sans toucher à aucun widget. C'est la fondation technique qui rend l'adaptation future possible.

15.3 Séparation UI / logique métier

Choix : architecture Clean Architecture + Riverpod.

Pourquoi pour l'accessibilité : l'état de l'application (loading, error, data) est géré dans les providers et non dans les widgets. Les états sont représentables de multiples façons sans dupliquer la logique. Un état `AsyncLoading` peut déclencher à la fois un indicateur visuel et une annonce TalkBack sans modifier le provider.

16. Accessibilité visuelle

16.1 Contrastes

La palette a été choisie pour garantir des ratios de contraste suffisants sur les éléments fonctionnels :

Combinaison	Ratio estimé	Standard WCAG AA
textPrimary (#444441) sur background (#F1EFE8)	~8:1	✓ Conforme (≥ 4.5:1)
textPrimary (#444441) sur surface (#FFFFFF)	~9:1	✓ Conforme
textSecondary (#888780) sur surface (#FFFFFF)	~4.5:1	✓ Limite conforme
textTertiary (#B4B2A9) sur surface (#FFFFFF)	~2.5:1	⚠ Non conforme (petits textes)
primary (#1D9E75) sur surface (#FFFFFF)	~4.6:1	✓ Conforme
textOnPrimary (#FFFFFF) sur primary (#1D9E75)	~4.6:1	✓ Conforme (boutons)

Note sur textTertiary : utilisée uniquement pour des métadonnées non critiques. Elle ne porte jamais d'information essentielle à la compréhension de l'écran.

16.2 Information jamais transmise par la couleur seule

Chaque état coloré est doublé d'un texte ou d'une icône :

- **RPE (effort post-séance)** : couleur (vert → orange → rouge) + label textuel ('Très facile' , 'Modéré' , 'Très difficile'). Un utilisateur daltonien comprend l'intensité via le texte.
- **Analyse IA nutrition** : `_AiAnalysisPanel` utilise couleur + icône (`Icons.check_circle` / `Icons.warning_amber_rounded`) + label ('Repas sûr' / 'Attention requise'). Trois canaux d'information redondants.
- **Barre de navigation** : onglet actif = couleur ET icône pleine (vs outline pour les inactifs).
- **Erreurs de connexion** : messages textuels affichés inline, pas seulement une bordure colorée.

16.3 Tailles de texte

Niveau	Taille	Usage	WCAG
Titre principal	22px	AppBar, titres d'écrans	✓
Titre secondaire	15–16px	Noms de recettes, sections	✓
Corps	13–14px	Contenus, labels	✓
Métadonnées	11–12px	Ingrédients, temps, prix	⚠ Petit
Labels uppercase	10px	Sections (MES PRÉFÉRENCES)	⚠ Non critique

Les tailles en 10–11px sont uniquement pour des informations contextuelles. Toute information nécessaire à une action est en 13px minimum.

17. Accessibilité motrice

17.1 Tailles des zones tactiles

`ElevatedButton` configuré avec `minimumSize: const Size(double.infinity, 44)`. Les `IconButton` Flutter ont une zone tactile de 48×48dp par défaut. Tous les boutons principaux respectent les recommandations Apple (44px) et Google Material (48dp).

17.2 Double entrée pour la capture photo

Deux méthodes de capture sont toujours proposées côte à côte :

[Photo – caméra live] [Galerie – image existante]

Un utilisateur avec un tremblement ou une faible mobilité peut utiliser la galerie à son rythme plutôt que de devoir déclencher la caméra au bon moment.

17.3 Slider RPE accessible

Le `Slider` Material natif garantit navigation clavier (flèches), annonce de la valeur par TalkBack/VoiceOver, et taille de cible tactile conforme.

17.4 Validation explicite des corrections

Le mode édition des macros (`MacroTableWidget`) nécessite un bouton `Valider` explicite. Pas de validation par perte de focus — cela évite les modifications accidentelles lors d'une navigation au clavier ou d'un scroll.

18. Accessibilité cognitive

18.1 Principe de moindre surprise

Navigation standard `BottomNavigationBar` à 4 onglets avec bouton central proéminent pour l'action principale. Ce pattern est immédiatement reconnaissable par tout utilisateur d'application mobile.

18.2 États de chargement toujours explicites

Action	Retour visuel	Retour textuel
Connexion	Spinner dans le bouton	Bouton désactivé
Upload nutrition	Spinner + écran résultat	Transition vers résultat
Génération menu	Spinner dans bouton	"Génération..."
Séance coach	Chronomètre animé	Décompte numérique

L'utilisateur sait toujours que quelque chose se passe.

18.3 Correction des erreurs IA autorisée

La reconnaissance nutritionnelle n'est pas imposée comme vérité absolue. Le tableau des macros dispose d'un mode édition permettant à l'utilisateur de corriger les valeurs. Cela répond à un besoin réel : les personnes avec un suivi médical précis de leur alimentation ne peuvent pas se permettre des valeurs approximatives.

18.4 Messages d'erreur explicites et actionnables

Les erreurs sont des phrases compréhensibles, pas des codes techniques. `_ErrorCard` affiche toujours un bouton `Réessayer`. L'utilisateur sait quoi faire ensuite.

18.5 Libellés en français, vocabulaire courant

L'interface est intégralement en français. Les termes potentiellement complexes sont contextualisés : - `RPE` est accompagné de son label (`'Modéré'` , `'Difficile'`) - Les allergènes sont en nom commun (`gluten` , `dairy` , `nuts`) - Les objectifs utilisent le vocabulaire courant (`Perte de poids` , `Prise de muscle` , `Maintien`)

19. Accessibilité des features spécifiques

19.1 Feature Nutrition

- **Double entrée** : caméra live + galerie
- **Compression automatique** : `ImageUtils.compress()` avant upload — utilisateurs sur connexion lente (3G, zones rurales) ne sont pas pénalisés
- **Timeout réseau** : `receiveTimeout: 30s` — l'application ne se fige pas
- **Analyse IA non-bloquante** : si le service échoue, les macros de base sont quand même affichées

19.2 Feature Menu IA

- **Slider calorique** avec pas de 100 kcal — pas de saisie clavier nécessaire

- **Allergies en FilterChips** — sélection par tap, évite les erreurs de frappe pour des informations médicalement importantes
- **Prix par personne affiché** — accessibilité économique
- **Ingrédients en chips** — liste visuelle scannée rapidement

19.3 Feature Coach

- **Label textuel RPE** — effort exprimé en mots, pas seulement en chiffre
- **Bouton "Passer"** — pas de contrainte obligatoire sur un utilisateur pressé ou fatigué
- **Timer en grande taille** — lisible à distance lors d'une séance

20. Accompagnement à l'adoption

20.1 Onboarding progressif par le design

L'application adopte un **onboarding implicite** sans écran de tutoriel forcé :

1. **Écran de connexion épuré** : logo, deux champs, un bouton. Aucune distraction.
2. **Onglet Nutrition comme écran d'accueil** : la feature la plus simple apparaît en premier. Le bouton  central est visuellement distinct et invite à l'action.
3. **Résultats immédiats** : l'utilisateur prend une photo et voit des macros en quelques secondes. La valeur de l'application est perceptible dès la première utilisation.
4. **Découverte par les onglets** : 4 onglets permettent une exploration séquentielle à son rythme.

20.2 Réduction de la friction à l'inscription

- Formulaire minimal : email + mot de passe uniquement
- Mot de passe masqué avec toggle (icône œil)
- Erreurs affichées inline, sans redirection

20.3 Confiance par la transparence

L'application ne dissimule pas ses limitations : - Si l'analyse IA échoue, les macros brutes sont quand même affichées - Le panneau IA n'apparaît que si l'analyse a réussi — pas de résultat inventé - Les ingrédients sont affichés en entier — l'utilisateur peut vérifier ce qu'il mange

Cette transparence est cruciale en santé : un utilisateur qui découvre que l'app lui a menti une fois ne lui fera plus confiance.

20.4 Contrôle utilisateur sur les données IA

La possibilité de corriger les macros détectées répond à un principe fondamental d'adoption : l'utilisateur doit avoir le sentiment de contrôler l'application. En santé particulièrement, les personnes suivant un régime médical strict ne peuvent pas se permettre des valeurs approximatives.

20.5 Système RPE comme rituel d'engagement

Le feedback post-séance (RPE 1–10) sert deux objectifs : 1. **Engagement** : créer un rituel post-séance qui renforce l'habitude d'utilisation 2. **Personnalisation future** : ces données permettront au Coach IA d'adapter les séances — l'utilisateur comprend que son feedback a une valeur concrète

21. Limites identifiées et axes d'amélioration

Limites actuelles

Limitation	Impact	Priorité
Pas de thème sombre	Difficile en basse luminosité / photosensibilité	Haute
<code>Semantics</code> non ajoutés manuellement	Widgets personnalisés non décrits aux lecteurs d'écran	Haute
Taille de police fixe (pas de <code>textScaleFactor</code>)	Agrandissement système non honoré	Moyenne
<code>textTertiary</code> sous le seuil WCAG AA	Ratio < 3:1 pour petits textes de métadonnées	Basse
Pas d'onboarding explicite	Utilisateurs non familiers peuvent manquer la valeur IA	Moyenne
Pas de mode hors ligne nutrition	Pas de logging repas sans connexion	Moyenne

Améliorations recommandées

1. `Semantics` sur les composants clés

```
Semantics(
  label: '1840 kilocalories consommées sur 2200, soit 84 pourcent',
  child: CircularProgressIndicator(value: 1840 / 2200),
)
```

2. Support du thème sombre

La palette `AppColors` étant centralisée, il suffit de créer `AppColors.dark` et de brancher `MediaQuery.platformBrightnessOf(context)` dans `app.dart`.

3. Respect de `textScaleFactor`

```
// Avant  
style: TextStyle(fontSize: 14)  
  
// Après  
style: TextStyle(fontSize: 14 * MediaQuery.textScaleFactorOf(context).clamp(1.0, 1.3))
```

4. Onboarding au premier lancement

3 slides maximum déclenchables via un flag `has_seen_onboarding` en `SharedPreferences` : - "Prenez en photo vos repas pour connaître leurs macros" - "Générez un menu personnalisé selon vos objectifs" - "Suivez vos séances sport et votre progression"